

1. Architecture Générale du Système

MEDICA+ repose sur une architecture client-serveur découplée et hautement modulaire. Les couches de présentation — le frontend web (médecin) et l'application mobile (patient) — sont totalement indépendantes et communiquent de manière asynchrone avec un backend API REST unique.

Tous les échanges s'effectuent au format standard JSON à travers des requêtes HTTP sécurisées de bout en bout par l'inclusion d'un jeton de sécurité JWT (JSON Web Token) transmis dans l'entête "Authorization" des requêtes.

2. Structure Globale du Code Source (Arborescence Clean)

Le projet adopte une séparation stricte des rôles par environnement applicatif :

```
MEDICA_PLUS/  
├── backend/           # Couche Serveur (Node.js & Express)  
│   ├── prisma/       # Configuration de la persistance de données  
│   └── src/  
│       ├── controllers/ # Logique métier et validation des requêtes  
│       └── routes/      # Routage et mappage des endpoints d'API  
├── frontend-web/      # Portail Médecin (React 18 / Vite)  
│   └── src/  
│       ├── components/ # Composants réutilisables (Sidebar, ProtectedRoute)  
│       ├── pages/      # Écrans métiers (Dashboard, Consultation, Agenda)  
│       └── services/    # Services de communication API (Axios, intercepteurs)  
├── mobile/            # Application Patient (React Native / Expo - Nohaeila)  
│   └── src/  
│       ├── components/ # UI Reutilisable (Boutons personnalisés, Cards de RDV)  
│       ├── screens/    # Écrans natifs (Connexion, Accueil, Prise de RDV, Profil)  
│       └── services/    # Appels API réseau (Fetch/Axios pour le flux mobile)
```

3. Modélisation de la Base de Données (Prisma ORM)

La base de données utilisée en environnement de développement est SQLite. La structure relationnelle est gérée par Prisma ORM, garantissant un typage fort et des migrations d'architecture fluides.

Modèles fondamentaux :

- User : Contient les informations de connexion (email, mot de passe hashé) et définit le rôle ("medecin" ou "patient"). Lié par une relation stricte 1:1 (@unique) vers les profils métiers.
- Medecin : Profil professionnel du praticien (nom, prénom, spécialité, adresse, téléphone).
- Patient : Profil personnel du patient (nom, prénom, date de naissance, Numéro de Sécurité Sociale - NSS).
- RendezVous : Table relationnelle modélisant le lien entre un médecin et un patient pour un créneau donné. Intègre les champs : date, heure, motif, et statut (PLANIFIE, CONFIRME, TERMINE, ANNULE).

- Dossier : Entité rattachée de manière unique à un Patient (1:1). Centralise les antécédents, les allergies (gérés par le patient via l'application mobile) et les notes cliniques (gérées par le médecin via l'interface web).
- Ordonnance & Rapport : Documents cliniques immuables créés par le médecin pour le compte d'un patient lors d'une consultation.
- Notification : Système d'alerte rattaché à un utilisateur avec suivi de lecture (booléen).

4. Spécifications de l'API REST

L'API expose des routes normalisées et sémantiques. L'accès aux données cliniques impose la fourniture d'un token JWT valide.

- Authentification :
 - POST /api/auth/register : Inscription utilisateur (Médecin/Patient).
 - POST /api/auth/login : Connexion, retourne le JWT.
- Médecins :
 - GET /api/medecins : Liste publique des professionnels (Annuaire consommé par le mobile).
 - GET /api/medecins/:id/patients : Liste restrictive des patients liés au médecin par un RDV.
 - PUT /api/medecins/profil : Mise à jour des informations professionnelles du médecin connecté.
- Rendez-vous :
 - GET /api/rdv : Liste des RDV de l'utilisateur connecté (filtrage automatique selon le rôle).
 - POST /api/rdv : Réservation d'un créneau (autorisé pour le rôle Patient uniquement via le mobile).
 - PUT /api/rdv/:id : Changement de statut d'un rendez-vous (Validation / Annulation).
- Dossier Médical & Documents :
 - GET /api/dossier/patient/:patientId : Lecture du dossier (réservé au médecin traitant).
 - POST /api/ordonnances : Création d'une ordonnance (Médecin uniquement).
 - POST /api/rapports : Enregistrement d'un compte-rendu clinique (Médecin uniquement).

5. Sécurité, Authentification et Confidentialité

- Chiffrement des Secrets : Les mots de passe des utilisateurs subissent un hashage asymétrique via l'algorithme bcrypt avec un facteur de coût (salt rounds) fixé à 10 avant d'être insérés en base de données.
- Middleware d'Interception (verifyToken) : Ce composant intercepte chaque requête vers une route protégée, extrait le jeton "Bearer" contenu dans le header Authorization, et valide son intégrité à l'aide de la clé secrète serveur (JWT_SECRET). La durée de validité du token est configurée à 24 heures. Si le jeton est valide, les données d'identité (id, rôle) sont injectées dans la variable de cycle de vie "req.user" pour permettre le contrôle d'accès au niveau des contrôleurs.

6. Spécifications des Couches Frontend et Mobile

- Interface Web Médecin :
 - Stack : React 18, Vite, TailwindCSS.
 - Identité visuelle : Couleur principale #4f8ef7 (Bleu médical), fond de page #f0f4ff, conteneurs avec bordures arrondies (border-radius: 14px) et ombre légère.
 - Sécurisation : Intercepteur Axios branché sur le localStorage pour injecter le jeton de sécurité. Protection d'accès via le composant "ProtectedRoute.jsx".

- Application Mobile Patient (Développée par Nohaeila) :
 - Stack : React Native via l'écosystème Expo (permettant un déploiement multiplateforme fluide).
 - Fonctionnalités clés : Création/Connexion de compte, sélection d'un praticien dans l'annuaire, prise de rendez-vous en temps réel, accès instantané aux ordonnances dématérialisées et mise à jour directe du volet déclaratif (antécédents, allergies).
 - Persistance : Stockage sécurisé du jeton JWT en local pour maintenir la session active de manière fluide à l'ouverture de l'application.

7. Guide de Déploiement et d'Exécution en Local

Pour déployer et initialiser l'environnement de développement, exécutez les séquences de commandes suivantes au sein de vos terminaux :

Terminal 1 : Initialisation du Serveur Backend

```
```bash
cd backend
npm install
npx prisma migrate dev
npm run dev
```